

Projet 7 : Classification automatique de reprises erronées

Morgane Bona-Pellissier, Salma Chatoui, Daria Tupikina et Sarah Yaya (Master 1 pluriTAL)

L'ensemble du projet est consultable sur le dépôt GitHub suivant : [crispyfunicular/classification_reprises_erronees](https://github.com/crispyfunicular/classification_reprises_erronees)

Résumé

L'objectif de ce projet est de **classifier automatiquement des reprises anaphoriques erronées en 3 classes** (problèmes grammaticaux, problèmes avec l'antécédent, problèmes avec la reprise) à partir d'un jeu de données annoté. Nous proposons un pipeline de Machine Learning tabulaire (features linguistiques et distances), avec une baseline (régression logistique, random forest et gradient boosting), une optimisation (GridSearch), et une analyse qualitative des erreurs via un rapport HTML.

Structure du dépôt (repères)

Les éléments principaux du dépôt sont les suivants (ce rapport détaille le fonctionnement du pipeline, tandis que ces repères facilitent la navigation et la reproductibilité) :

- `pipeline.py` : socle de base (construction de la matrice, prétraitement, modèles baseline).
- `amelioration.py` : optimisation (`GridSearchCV`) + interprétabilité (importance des variables) + génération du rapport d'erreurs.
- `visualisation.py` : export du rapport HTML qualitatif (analyse des erreurs).
- `feature_engineering.py` : enrichissement du dataset (extraction morphologique, variables d'accord).
- `evaluation_features.py` : évaluation de l'impact du feature engineering (comparaison des performances).
- `dataset_erreurs_reprises.xlsx` : données annotées de départ.
- `dataset_enrichi.xlsx` : données enrichies (générées par le feature engineering).
- `rapport.md` : rapport technique.
- `requirements.txt` : dépendances Python.

Élaboration du pipeline

Étape 1 : création de la matrice (`construire_matrice()`)

Consigne : *Récupérer un fichier csv auprès de Vanessa Gaudray Bouju. Le fichier contient : la reprise, son antécédent, le contexte*

antérieur, le type de reprise, et d'autres infos. L'objectif est de classer les reprises erronées en 3 classes (problèmes grammaticaux, problèmes avec l'antécédent, problèmes avec la reprise).

Postulat : chaque type d'erreur est corrélé à des variables spécifiques

- **Problèmes d'antécédent** : Le modèle se concentrera sur l'ambiguïté et l'éloignement (GN_concurrents, GN_concurrents_compatibles, Distance_phrases).
- **Problèmes de reprise** : Le modèle se concentrera sur la nature du mot et le sens (Type_pronom, Definitude_GN, Similarite_reprise_antecedent).
- **Problèmes grammaticaux** : Le modèle se concentrera sur la syntaxe (Fonction_reprise, Fonction_antecedent, Distance_mots).

Lors de cette première étape, la fonction `construire_matrice()` effectue la construction de la matrice initiale :

- **Chargement des données brutes** depuis le format Excel originel.
- **Création de la variable cible (Target)** en regroupant les valeurs initiales (TypeErreur1) en 3 classes de problèmes distinctes (Problème_Grammatical, Problème_Antecedent, Problème_Reprise) pour simplifier et cibler la prédiction.
- **Suppression des valeurs** manquantes au niveau de la variable cible.
- **Sélection des features** à exploiter en distinguant explicitement les features numériques (distances, similarité sémantique, concurrences, etc.) et les features catégorielles (fonctions syntaxiques, types de pronoms, définitude, etc.).
- **Séparation** de la matrice des features **X** et du vecteur attendu **y**, transmis directement à l'étape suivante.

Jeu de données et lien avec les consignes La consigne mentionne un fichier au format CSV. Dans notre dépôt, les données sont stockées dans `dataset_erreurs_reprises.xlsx` (même contenu exploitable). Les champs exigés par la consigne sont couverts par les colonnes suivantes :

- **Reprise** : TexteErreur
- **Antécédent** : Antecedent
- **Contexte antérieur** : Contexte
- **Type de reprise** : TypeReprise
- **Autres infos** : distances (Distance_*), traits syntaxiques (Fonction_*), informations sur les pronoms (Type_pronom, etc.)

La variable cible utilisée pour la classification est `TypeErreur1`, regroupée en 3 classes.

Étape 2 : préparation des données (`pretraiter()`)

Consigne : *Diviser le corpus en train/test.*

Action technique : Nous avons utilisé un `ColumnTransformer` (via Scikit-Learn) pour appliquer le One-Hot Encoding uniquement sur les variables catégorielles pertinentes, normaliser les distances, et exclure le texte brut de nos modèles de Machine Learning classiques.

À partir de la matrice générée, la fonction `pretraiter()` prépare les données pour la modélisation à venir : - **Construction d'un ColumnTransformer (Pipeline de Pré-traitement)** permettant des traitements spécifiques selon le type de feature : - *Features numériques* : Traitement par imputation de la médiane (complète les données manquantes de manière robuste aux valeurs aberrantes), puis normalisation (`StandardScaler`). - *Features catégorielles* : Imputation des données manquantes par une valeur dédiée ("absent" — puisque cette information peut jouer un rôle utile pour le modèle) suivie de leur encodage en variables muettes (`OneHotEncoder`). - **Séparation stratifiée du jeu de données (Train/Test Split)** (80%/20%), ce qui conserve la même proportion des classes cible entre les exemples dédiés à l'apprentissage (Train) et ceux pour l'évaluation finale (Test). - **Application stricte du préprocesseur** (ajustement ou `fit`) exclusivement sur le corpus d'entraînement afin d'éviter toute fuite d'informations vers le corpus de test (Data Leakage). Les deux ensembles sont ensuite passés en transformation (`transform`). - Les données prétraitées sont transmises directement à l'étape suivante.

Étape 3 : entraînement et évaluation des modèles baseline (`evaluer_baselines()`)

La fonction `evaluer_baselines()` établit les premières performances de référence en entraînant trois modèles classiques sur les données préparées à l'étape 2 : - **Définition de trois modèles de référence (Baseline)**, en appliquant un poids proportionnel aux classes (`class_weight='balanced'`) pour compenser leur déséquilibre : - *Régression Logistique* : modèle linéaire et interprétable. - *Random Forest* : modèle ensembliste et non-linéaire basé sur des arbres de décision. - *Gradient Boosting* : modèle ensembliste plus fort qui combine plusieurs arbres de décision (il les applique uns après les autres) et qui fait plus attention aux erreurs faites sur chaque étape. - **Entraînement et évaluation sur le jeu de test** : chaque modèle est entraîné sur les données Train (`fit`), puis évalué sur le Test, produisant l'accuracy, un rapport détaillé de classification (précision, rappel, F1-score par classe) ainsi qu'une matrice de confusion. - **Validation croisée (Cross-validation 5-fold)** : comme le jeu de données est restreint (299 exemples), le script effectue une validation croisée sur l'ensemble du jeu de données (5 partitions) afin d'obtenir une estimation plus robuste et stabilisée des performances, en mesurant le F1-score macro.

Étape 4 : Optimisation et Analyse Qualitative des Erreurs (`amelioration.py`)

Afin de maximiser les performances de nos algorithmes et rendre le modèle intelligible pour l'analyse linguistique, nous avons créé un script dédié à l'optimisation,

`amelioration.py`. Ce script procède aux étapes suivantes :

1. **Optimisation des hyperparamètres (Tuning)** : Utilisation de `GridSearchCV` (validation croisée sur 5 folds) pour tester méthodiquement une grille exhaustive de paramètres sur la *régression logistique* (paramètres de régularisation `C`, `solver`), le *random forest* (profondeur maximale, nombre d'arbres `n_estimators`, critère de séparation) et le *gradient boosting* (ajustement du taux d'apprentissage `learning_rate`, du nombre d'itérations `n_estimators` et de la profondeur des arbres `max_depth`). Le but est de trouver la configuration maximisant le F1-score au-delà de la baseline existante (implémentée dans `optimiser_modeles()`).
2. **Interprétabilité (Feature Importance)** : Une fois le meilleur modèle non-linéaire (Random Forest) entraîné, nous extrayons l'importance relative de chaque caractéristique (Feature Importance) via `afficher_feature_importance()`. Cette étape permet de justifier notre **postulat initial** : le modèle se base effectivement majoritairement sur les distances (en caractères, mots, phrases) et sur la similarité sémantique plutôt que de deviner au hasard.
3. **Analyse Qualitative des Erreurs** : L'évaluation mathématique (Accuracy, F1-score) est indispensable mais insuffisante en NLP structuré. Pour répondre au besoin qualitatif, le script exporte un document de diagnostic (`analyse_erreurs.html`) via `exporter_erreurs_html()`. Ce rapport confronte dynamiquement la classe attendue ("Vraie Classe") et l'erreur du classifieur ("Classe Prédite"). Il intègre le contexte textuel brut avec surlignage interactif de l'antécédent et de la reprise, facilitant l'exploration et la formulation de nouvelles hypothèses linguistiques.

Étape 5 : Comparaison systématique des configurations (`scoring.py`)

Pour centraliser les résultats et comparer plusieurs configurations, nous ajoutons `scoring.py` qui évalue 4 combinaisons dataset $\times$ méthode :

- Dataset **base** vs dataset **enrichi** (généré par `feature_engineering.py` via `enrichir_dataset()`)
- Méthode **baseline** vs **GridSearch**

Le script génère automatiquement `resultats_scoring.csv` (F1-macro, F1-weighted, balanced accuracy) afin de faciliter la rédaction du rapport.

Discussion des résultats

Tableau récapitulatif des résultats

Quatre combinaisons dataset $\times$ méthode ont été évaluées, produisant douze configurations au total (trois modèles par combinaison). Les résultats sont rassemblés dans le tableau suivant.

Dataset	Méthode	Modèle	F1_macro	F1_weighted	Balanced_accuracy
Base	Baseline	LogisticRegression	0.5934	0.5748	0.6185
Base	Baseline	RandomForest	0.6236	0.6155	0.6296
Base	Baseline	GradientBoosting	0.6379	0.6293	0.6432
Base	GridSearch	LogisticRegression	0.5741	0.5611	0.5963
Base	GridSearch	RandomForest	0.6600	0.6501	0.6580
Base	GridSearch	GradientBoosting	0.5469	0.5355	0.5432
Enrichi	Baseline	LogisticRegression	0.7206	0.7151	0.7358
Enrichi	Baseline	RandomForest	0.6801	0.6938	0.6741
Enrichi	Baseline	GradientBoosting	0.7613	0.7663	0.7568
Enrichi	GridSearch	LogisticRegression	0.6648	0.6646	0.6642
Enrichi	GridSearch	RandomForest	0.6657	0.6774	0.6617
Enrichi	GridSearch	GradientBoosting	0.6839	0.6964	0.6778

Ces résultats sont générés automatiquement par `scoring.py` et enregistrés dans `resultats_scoring.csv`.

Choix des métriques et justification

Trois métriques complémentaires ont été retenues pour évaluer les modèles, en réponse au déséquilibre des classes observé dans le jeu de données (133 problèmes d'antécédent / 92 problèmes de reprise / 74 problèmes grammaticaux).

- Le **F1-macro** est la métrique de référence de notre analyse : il calcule la moyenne non pondérée du F1-score de chaque classe, donnant ainsi le même poids aux trois catégories d'erreurs quelle que soit leur fréquence. Il pénalise fortement un modèle qui ignorerait la classe minoritaire (problèmes grammaticaux), ce qui correspond précisément à notre objectif de détection équilibrée.
- Le **F1-weighted** pondère le F1-score par le nombre d'exemples de chaque classe. Il reflète la performance globale sur l'ensemble du corpus et tend à être plus élevé que le F1-macro, car les classes majoritaires, mieux apprises, tirent la moyenne vers le haut.
- La **balanced accuracy** (moyenne des rappels par classe) constitue un second indicateur de robustesse face au déséquilibre et confirme les tendances observées avec le F1-macro.

Le feature engineering : levier décisif

L'apport le plus significatif ne vient pas du réglage des hyperparamètres, mais de l'**enrichissement du jeu de données**. En ajoutant des variables morphologiques (genre, nombre, type de pronom) et des indicateurs d'accord (Match_genre, Match_nombre), le F1-macro de la régression logistique progresse nettement entre le dataset base (0,593) et le dataset enrichi (0,721) en baseline. Le Random Forest progresse également sur le dataset base avec tuning (0,660), mais reste en dessous de la meilleure configuration observée sur le dataset enrichi. Le Gradient Boosting montre aussi une croissance importante du score F1 grâce à l'enrichissement du dataset (0,64 sur le dataset d'origine

et 0,76 sur celui enrichi avec la méthode baseline). Ce résultat confirme que les features de base (distance caractères/mots et fonctions syntaxiques) ne capturent pas suffisamment les patterns discriminants entre les trois classes d'erreurs. Les variables morphologiques fournissent une information linguistiquement plus pertinente pour distinguer une erreur grammaticale d'un problème de reprise, car elles encodent directement les contraintes d'accord qui définissent ces deux types d'erreurs.

L'effet du GridSearch selon le dataset

Le GridSearch ne produit pas le même effet selon la richesse du dataset. Sur le dataset base, le GridSearch n'apporte pas d'amélioration pour la régression logistique (0,593 -> 0,574), mais améliore le Random Forest (0,624 -> 0,660). En revanche, il réduit significativement le résultat du Gradient Boosting (0,64 -> 0,54). Sur le dataset enrichi, le GridSearch dégrade les scores par rapport à la baseline pour les trois modèles (régression logistique : 0,721 -> 0,665 ; random forest : 0,680 -> 0,666 ; gradient boosting : 0,76 -> 0,69), ce qui suggère un surajustement sur l'entraînement malgré la validation croisée interne au GridSearch.

La régression logistique surpasse le Random Forest

Comme attendu, la meilleure configuration observée correspond à un modèle ensembliste itératif (gradient boosting) sur le dataset enrichi en **baseline**. Un résultat marquant ressort de cette comparaison : la Régression Logistique se classe juste après, affichant des résultats supérieurs à ceux du Random Forest. Le Random Forest, pourtant plus complexe et réputé pour sa capacité à capturer des interactions non-linéaires, ne le dépasse pas ici. Deux hypothèses expliquent ce résultat. D'une part, les variables morphologiques ajoutées (Match_genre, Match_nombre, Est_pronom) créent des signaux proches de frontières de décision quasi-linéaires (par exemple un désaccord en genre/nombre), que la régression logistique exploite efficacement. D'autre part, le dataset reste petit (299 exemples) : des modèles plus complexes peuvent sur-apprendre plus facilement.

Cohérence entre les métriques

Les trois métriques sont globalement cohérentes : le classement des configurations ne change pas selon qu'on utilise le F1-macro, le F1-weighted ou la balanced accuracy. On note toutefois que le F1-weighted est systématiquement supérieur au F1-macro (de 0.02 à 0.03 points), ce qui confirme que les classes majoritaires sont mieux apprises que les classes rares. Cet écart persistant, même dans la meilleure configuration, indique une marge de progression spécifique sur la classe « problèmes grammaticaux ».

Conclusion

Ce projet a permis de construire un pipeline complet de classification automatique des reprises anaphoriques erronées en français, depuis la constitution du jeu de données jusqu'à l'analyse qualitative des erreurs du modèle. Quatre configurations ont été comparées de manière systématique, croisant deux datasets (base et enrichi) avec deux stratégies d'optimisation (baseline et GridSearch) sur trois algorithmes (régression logistique, Random Forest et Gradient Boosting). Le résultat principal est sans ambiguïté : l'enrichissement en traits morphologiques est un levier majeur de performance. La meilleure configuration observée est le **Gradient Boosting en baseline sur le dataset enrichi** (F1-macro 0,761), et l'analyse qualitative (rapport HTML généré) reste indispensable pour interpréter les erreurs résiduelles et guider de futurs raffinements. Ce travail présente néanmoins plusieurs limites. Le jeu de données annoté est de taille réduite (299 exemples), ce qui rend les estimations de performance sensibles au split train/test et limite la capacité des modèles complexes à généraliser. Par ailleurs, les performances restent modestes en valeur absolue : même un F1-macro autour de 0,76 indique qu'une fraction non négligeable des erreurs est encore mal classée, notamment dans la classe minoritaire « problèmes grammaticaux ».

Annexe : Reproductibilité (commandes)

Installation

```
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Exécution des scripts

Scripts (liens) : - pipeline.py - amelioration.py - feature_engineering.py
- scoring.py

```
./venv/bin/python pipeline.py
./venv/bin/python amelioration.py
./venv/bin/python feature_engineering.py
./venv/bin/python scoring.py
```

Sorties attendues (artefacts)

- `amelioration.py` génère un rapport HTML de diagnostic des erreurs (`analyse_erreurs.html`) permettant une analyse qualitative (contexte, antécédent, reprise, vraie classe vs classe prédite).
- `feature_engineering.py` produit un dataset enrichi (`dataset_enrichi.xlsx`) utilisé dans les comparaisons de performances.